

PAI/1

Protocolo Asíncrono de Imágenes

PAI/1 es un protocolo de aplicación propio para que el navegador solicite una región de una imagen y reciba esa vista dividida en fragmentos pequeños (chunks). El navegador no necesita descargar el archivo original completo.

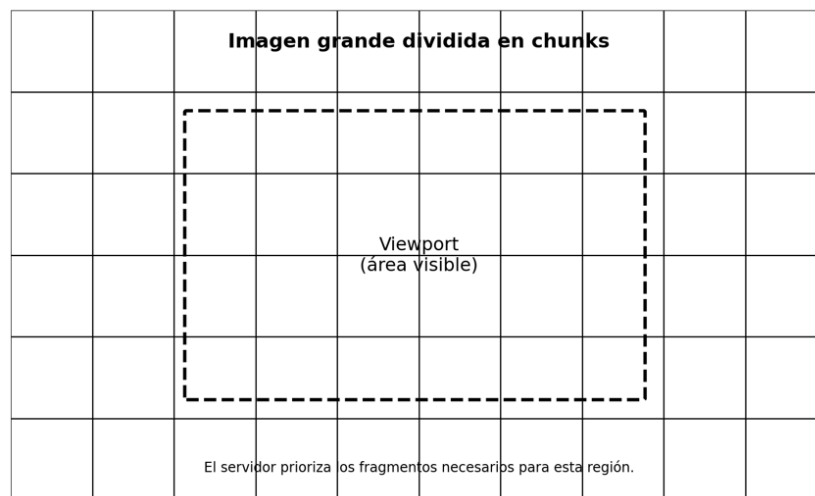
La implementación combina mensajes binarios, identificación de cada vista mediante generationId, una ventana de envío de chunks pendientes de confirmación, reintentos cuando no llega un ACK y reutilización opcional de fragmentos que el navegador ya conserva en su caché.

PAI/1 define mensajes y políticas de aplicación sobre WebSocket. Su ventana y sus CHUNK_ACK indican cuándo el navegador terminó de procesar cada pieza; TCP conserva por separado su transporte confiable y control de congestión.

Problema para resolver

Una imagen de decenas de gigabytes no debe enviarse completa al navegador. Además, aunque el navegador pueda ampliar o mover la imagen, solo necesita mostrar una región de ella en un momento concreto.

- El cliente indica qué imagen quiere, el nivel de zoom, el centro de la vista y el tamaño del área visible.
- El servidor calcula qué región de la imagen original corresponde a esa vista.
- La región se divide en chunks y se procesan solo los fragmentos requeridos.
- Los chunks se envían de forma controlada y el navegador los dibuja en un canvas.
- Cuando llega una nueva solicitud de vista, la generación anterior se cancela para no seguir trabajando en una vista que ya quedó obsoleta.



Capas y componentes

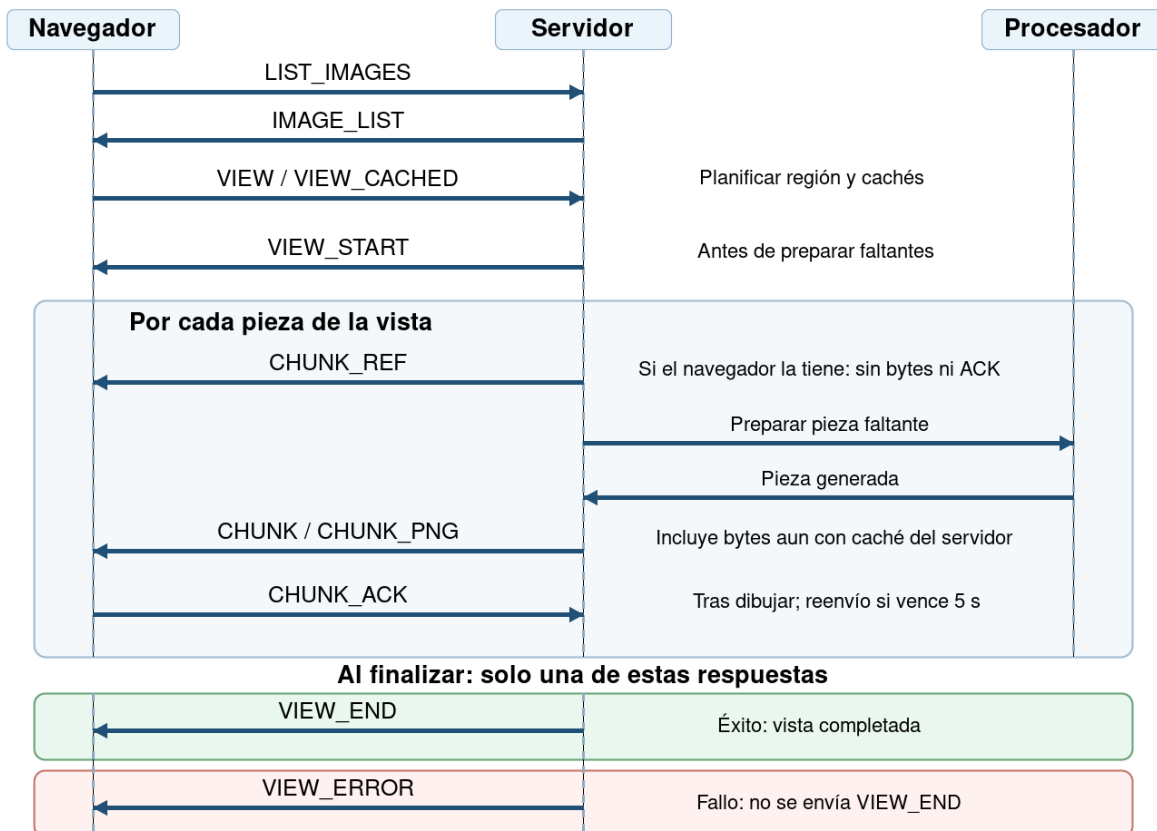


Componente	Responsabilidad
HTTP	Entrega index.html, CSS y módulos JavaScript del visor.
WebSocket /pai	Mantiene una conexión bidireccional para enviar mensajes binarios PAI/1 en ambas direcciones.
PAI/1	Define la firma, versión, operación y estructura de cada mensaje.
AsyncHttpServer	Acepta conexiones, decodifica frames WebSocket, valida mensajes PAI y coordina la sesión del cliente.
ViewCoordinator	Mantiene una generación activa por cliente y cancela la tarea anterior al recibir una nueva VIEW.
ViewProcessor	Calcula la región y el plan de chunks, consulta la caché y coordina la generación de los que faltan.
VipsChunkRenderer / preparación	Usa herramientas de imagen para producir los fragmentos regionales; las imágenes gigantes pueden requerir preparación previa.
ChunkCache	Conserva bytes de chunks en una caché GreedyDual-Size compartida y limitada por bytes.
ViewFlowControl	Aplica una ventana de hasta cuatro chunks sin ACK, temporizador de reenvío y límite de reintentos.
Cliente web	Envía VIEW, procesa respuestas, valida generaciones y metadatos, decodifica imágenes y dibuja en canvas.
Caché de bitmaps del navegador	Conserva imágenes ya decodificadas y permite que el cliente anuncie fragmentos reutilizables mediante VIEW_CACHED.

Ciclo de una conexión

1. El navegador solicita la página por HTTP.
2. JavaScript abre un WebSocket con el endpoint /pai.
3. El cliente envía LIST_IMAGES y el servidor responde IMAGE_LIST con los metadatos de las imágenes disponibles.
4. El usuario elige una imagen o cambia zoom/posición; el navegador envía VIEW. Si anuncia fragmentos ya almacenados, usa VIEW_CACHED.
5. El servidor valida el mensaje y crea un ViewRequest.
6. ViewCoordinator activa la nueva generación y cancela la anterior si aún estaba ejecutándose.
7. El servidor calcula la región y el plan de chunks, consulta la caché y envía VIEW_START. Después prepara los chunks faltantes y los entrega conforme están disponibles.
8. Para cada posición, el servidor envía CHUNK/CHUNK_PNG o, si el cliente anunció una pieza reutilizable, CHUNK_REF.
9. El cliente valida y dibuja los chunks de imagen; después de procesar y dibujar un CHUNK, envía CHUNK_ACK.
10. El servidor espera a que no queden chunks de imagen pendientes de ACK antes de enviar VIEW_END.
11. Si la vista no puede completarse, el servidor puede enviar VIEW_ERROR sin cerrar necesariamente el WebSocket.

Secuencia PAI/1



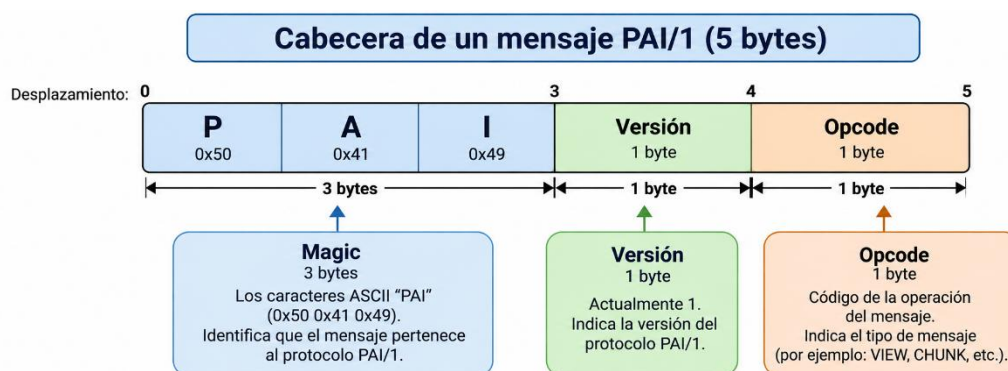
Formato común de los mensajes PAI/1

Todos los mensajes PAI comienzan con una cabecera fija de cinco bytes. Los campos numéricos de PAI se codifican en orden big-endian, es decir, primero se transmite el byte más significativo.

Desplazamiento	Tamaño	Campo	Significado
0	3 bytes	Magic	Los caracteres ASCII PAI.
3	1 byte	Versión	Actualmente 1.
4	1 byte	Opcode	Código de la operación del mensaje.

Los mensajes viajan dentro de frames binarios de WebSocket. La cabecera PAI identifica el protocolo de aplicación; no sustituye las cabeceras propias de WebSocket, TCP o IP.

Magic son los tres bytes ASCII PAI (0x50 0x41 0x49). Junto con la versión y el opcode, permite rechazar mensajes ajenos o incompatibles; no cifra ni autentica la conexión.



Estructura binaria y desplazamientos de los mensajes PAI/1

Todos los mensajes PAI/1 comienzan con una cabecera fija de 5 bytes: 3 bytes para Magic, 1 byte para la versión y 1 byte para el Opcode. Los campos numéricos se codifican en orden big-endian.

VIEW_START

VIEW_START tiene un tamaño fijo de 45 bytes. El mensaje contiene la información necesaria para describir la vista que será entregada al cliente.

Desplazamiento	Tamaño	Campo	Descripción
0	3 bytes	Magic	Cadena ASCII PAI
3	1 byte	Versión	Versión del protocolo, actualmente 1
4	1 byte	Opcode	Código VIEW_START
5	8 bytes	generationId	Identificador de la generación de la vista
13	4 bytes	viewportWidth	Ancho del viewport
17	4 bytes	viewportHeight	Alto del viewport
21	4 bytes	zoomIndex	Nivel de zoom
25	4 bytes	regionX	Coordenada X de la región
29	4 bytes	regionY	Coordenada Y de la región
33	4 bytes	regionWidth	Ancho de la región
37	4 bytes	regionHeight	Alto de la región
41	4 bytes	chunkCount	Cantidad de chunks de la vista

Tamaño total: 45 bytes

CHUNK y CHUNK_PNG

CHUNK y CHUNK_PNG utilizan la misma estructura de metadatos. La diferencia es el formato de los bytes de imagen: CHUNK transporta JPEG y CHUNK_PNG transporta PNG. Los metadatos ocupan 45 bytes, seguidos por la imagen codificada.

Desplazamiento	Tamaño	Campo	Descripción
0	3 bytes	Magic	Cadena ASCII PAI
3	1 byte	Versión	Versión del protocolo
4	1 byte	Opcode	CHUNK o CHUNK_PNG
5	8 bytes	generationId	Generación de la vista
13	4 bytes	chunkIndex	Índice del chunk
17	4 bytes	column	Columna del chunk
21	4 bytes	row	Fila del chunk
25	4 bytes	canvasX	Posición X dentro del canvas
29	4 bytes	canvasY	Posición Y dentro del canvas
33	4 bytes	width	Ancho de salida
37	4 bytes	height	Alto de salida
41	4 bytes	imageLength	Longitud de la imagen codificada
45	Variable	imageData	Bytes JPEG o PNG

Metadatos: 45 bytes. Tamaño total: 45 + imageLength bytes.

CHUNK_ACK

CHUNK_ACK confirma que el cliente procesó un chunk concreto. Su tamaño total es de **17 bytes**.

Desplazamiento	Tamaño	Campo	Descripción
0	3 bytes	Magic	Cadena ASCII PAI
3	1 byte	Versión	Versión del protocolo
4	1 byte	Opcode	Código CHUNK_ACK
5	8 bytes	generationId	Generación de la vista
13	4 bytes	chunkIndex	Índice del chunk confirmado

Tamaño total: 17 bytes.

CHUNK_REF

CHUNK_REF se utiliza cuando el navegador ya conserva el bitmap correspondiente en su caché. Por esta razón, el mensaje contiene únicamente los metadatos necesarios para identificar y posicionar el fragmento y no contiene bytes de imagen. Su tamaño total es de 41 bytes.

Desplazamiento	Tamaño	Campo	Descripción
0	3 bytes	Magic	Cadena ASCII PAI
3	1 byte	Versión	Versión del protocolo
4	1 byte	Opcode	Código CHUNK_REF
5	8 bytes	generationId	Generación de la vista
13	4 bytes	chunkIndex	Índice del chunk

17	4 bytes	column	Columna del chunk
21	4 bytes	row	Fila del chunk
25	4 bytes	canvasX	Posición X dentro del canvas
29	4 bytes	canvasY	Posición Y dentro del canvas
33	4 bytes	width	Ancho del fragmento
37	4 bytes	height	Alto del fragmento

Tamaño total: 41 bytes.

VIEW_END

VIEW_END indica que la vista terminó. Sus 25 bytes contienen generación, número de piezas y suma de bytes codificados de las imágenes de la vista, contando cada chunk una vez.

Desplazamiento	Tamaño	Campo	Descripción
0	3 bytes	Magic	Cadena ASCII PAI
3	1 byte	Versión	Versión del protocolo
4	1 byte	Opcode	Código VIEW_END
5	8 bytes	generationId	Generación de la vista
13	4 bytes	chunkCount	Cantidad total de piezas entregadas
17	8 bytes	imageBytes	Suma de payloads de imagen únicos

Tamaño total: 25 bytes.

Interpretación de los contadores de VIEW_END

chunkCount representa todas las piezas entregadas para la vista, incluyendo tanto los chunks que transportaron bytes de imagen (CHUNK o CHUNK_PNG) como las referencias CHUNK_REF.

imageBytes suma los payloads JPEG/PNG de los CHUNK de la vista una vez por pieza. No incluye CHUNK_REF, cabeceras PAI/WebSocket/TCP ni posibles retransmisiones; por eso no es una medición de bytes totales en la red.

VIEW_CACHED

VIEW_CACHED utiliza los mismos campos iniciales de VIEW y agrega una lista de chunks que el navegador conserva en su caché. Cada entrada de la lista ocupa 16 bytes: columna, fila, ancho y alto, con 4 bytes para cada campo.

Campo	Tamaño	Descripción
Cabecera PAI	5 bytes	Magic, versión y opcode
Campos de VIEW	30 bytes fijos + N bytes de imageId	generationId, zoomIndex, centro, viewport, imageIdLength (2 bytes) e ID UTF-8 de N bytes
cachedCount	2 bytes	Cantidad de chunks anunciados
Entradas	16 × cachedCount bytes	Información de cada chunk en caché

El valor de cachedCount debe estar entre 1 y 128; cero no es válido. Si N es la longitud del ID UTF-8 y $K = \text{cachedCount}$, el tamaño total es $35 + N + 2 + 16 \times K$ bytes. El campo imageIdLength ocupa los bytes 33–34 y ya forma parte de los 30 bytes fijos posteriores a la cabecera PAI. cachedCount ocupa los bytes 35 + N y 36 + N; la entrada i (iniciando en 0) comienza en el byte $37 + N + 16 \times i$. Cada entrada contiene:

Desplazamiento relativo	Tamaño	Campo
0	4 bytes	column
4	4 bytes	row
8	4 bytes	width
12	4 bytes	height

VIEW (opcode 1)

Desplazamiento	Tamaño	Campo	Descripción
0-4	5	Cabecera PAI	Magic, versión y opcode VIEW
5-12	8	generationId	Generación creciente por sesión
13-16	4	zoomIndex	Nivel solicitado
17-20	4	centerX	Centro X en el original
21-24	4	centerY	Centro Y en el original
25-28	4	viewportWidth	Ancho del área visible
29-32	4	viewportHeight	Alto del área visible
33-34	2	imageIdLength	N: longitud del ID UTF-8
35-(34 + N)	N	imageId	Identificador UTF-8; 1-128 bytes

Tamaño total de VIEW: 35 + N bytes. Los enteros usan big-endian.

LIST_IMAGES (opcode 2)

Desplazamiento	Tamaño	Campo	Descripción
0-4	5	Cabecera PAI	Magic, versión y opcode LIST_IMAGES

Tamaño total: 5 bytes; no hay cuerpo adicional.

IMAGE_LIST (opcode 3)

Desplazamiento	Tamaño	Campo	Descripción
0-4	5	Cabecera PAI	Magic, versión y opcode IMAGE_LIST
5-6	2	imageCount	Número de entradas; máximo 1024
q + 0	2	idLength	N: longitud del ID UTF-8
q + 2	N	imageId	ID de la imagen
q + 2 + N	2	nameLength	M: longitud del nombre UTF-8
q + 4 + N	M	name	Nombre del archivo
q + 4 + N + M	4	width	Ancho original
q + 8 + N + M	4	height	Alto original
q + 12 + N + M	8	sizeBytes	Tamaño original en bytes

q = 7 para la primera entrada. Cada entrada ocupa 20 + N + M bytes; la siguiente comienza en q + 20 + N + M.

VIEW_ERROR (opcode 8)

Desplazamiento	Tamaño	Campo	Descripción
0-4	5	Cabecera PAI	Magic, versión y opcode VIEW_ERROR
5-12	8	generationId	Generación fallida
13-14	2	messageLength	M: longitud del texto UTF-8
15-(14 + M)	M	message	Descripción de hasta 240 bytes

Tamaño total: 15 + M bytes. El servidor intenta notificar el error; una generación fallida no envía VIEW_END.

Tabla de operaciones

Opcode	Nombre	Dirección habitual	Propósito
1	VIEW	Cliente → servidor	Solicita una vista sin lista de fragmentos reutilizables.
2	LIST_IMAGES	Cliente → servidor	Solicita el catálogo de imágenes.
3	IMAGE_LIST	Servidor → cliente	Devuelve ID, nombre, dimensiones y tamaño de cada imagen.
4	VIEW_START	Servidor → cliente	Anuncia la generación, viewport, zoom, región y cantidad de chunks.
5	CHUNK	Servidor → cliente	Entrega un fragmento codificado como JPEG.
6	VIEW_END	Servidor → cliente	Confirma que la vista terminó y resume cantidad de chunks y bytes de imagen.
7	CHUNK_PNG	Servidor → cliente	Entrega un fragmento codificado como PNG; se utiliza para los niveles de detalle que lo requieren.
8	VIEW_ERROR	Servidor → cliente	Informa que una vista no se pudo completar.
9	CHUNK_ACK	Cliente → servidor	Confirma la recepción y el procesamiento de un chunk de imagen.
10	VIEW_CACHED	Cliente → servidor	Solicita una vista y anuncia hasta 128 fragmentos que ya conserva.
11	CHUNK_REF	Servidor → cliente	Referencia un fragmento anunciado como disponible; no incluye los bytes de la imagen.

Mensajes de solicitud

VIEW

VIEW identifica la imagen y describe el encuadre deseado. El cuerpo fijo contiene generationId (8 bytes), zoomIndex (4), centerX (4), centerY (4), viewportWidth (4), viewportHeight (4), la longitud del ID (2) y el ID UTF-8. El tamaño mínimo antes del texto del ID es de 35 bytes incluyendo la cabecera PAI.

Campo	Tamaño	Uso
generationId	8 bytes	Identificador creciente de la solicitud de vista.
zoomIndex	4 bytes	Nivel de zoom que el visor solicita.
centerX / centerY	4 + 4 bytes	Centro deseado de la vista en coordenadas de la imagen.
viewportWidth / viewportHeight	4 + 4 bytes	Dimensiones del área visible del navegador.
imageIdLength	2 bytes	Longitud del ID en bytes UTF-8.
imageId	Variable	Identificador de imagen; el codec limita el ID a 128 bytes y valida caracteres permitidos.

El identificador de imagen se valida con el patrón de caracteres a-z, 0-9, guion bajo y guion. Esto evita que un nombre de archivo arbitrario se trate directamente como una ruta.

VIEW_CACHED

VIEW_CACHED usa los mismos campos de VIEW y añade una lista de piezas que el navegador afirma conservar en su caché de bitmaps. La lista lleva un contador de 2 bytes y hasta 128 entradas. Cada entrada contiene columna, fila, ancho y alto, con 4 bytes por campo (16 bytes por entrada).

El servidor puede responder con CHUNK_REF para una de esas piezas. El mensaje de referencia contiene los datos geométricos y el índice de la pieza, pero no la imagen codificada. El cliente verifica que la referencia corresponda a una pieza anunciada y que el bitmap siga en su caché.

LIST_IMAGES e IMAGE_LIST

LIST_IMAGES es un mensaje que contiene únicamente la cabecera PAI. IMAGE_LIST empieza con un contador de 2 bytes y luego incluye, por cada imagen, el ID y el nombre como texto UTF-8 precedido por su longitud de 2 bytes, el ancho (4 bytes), el alto (4 bytes) y el tamaño del archivo original en bytes (8 bytes). El cliente valida las longitudes, los metadatos y que no sobren bytes al final.

Mensajes de respuesta de una vista

VIEW_START

VIEW_START abre la respuesta lógica de una vista. Incluye generationId (8 bytes), viewportWidth y viewportHeight (4 bytes cada uno), zoomIndex (4), la región calculada en la imagen original (x, y, ancho y alto; 4 bytes cada campo) y la cantidad total de chunks (4 bytes).

CHUNK y CHUNK_PNG

Ambas operaciones tienen la misma estructura de metadatos; cambia el tipo de imagen adjunta. CHUNK lleva JPEG y CHUNK_PNG lleva PNG. Después de la cabecera PAI, el mensaje contiene generationId (8), índice (4), columna (4), fila (4), posición X e Y dentro del canvas (4 cada una), ancho y alto de salida (4 cada uno), longitud de la imagen (4) y finalmente los bytes JPEG o PNG.

El cliente valida que la longitud declarada coincida con el tamaño del mensaje y que los bytes tengan una firma de JPEG o PNG válida. El límite de tamaño de imagen codificada por chunk definido en el cliente es 8 MiB; la cantidad máxima de chunks por vista definida en el cliente es 4096.

CHUNK_ACK

CHUNK_ACK confirma un chunk concreto. Después de la cabecera de 5 bytes, lleva generationId de 8 bytes e índice de chunk de 4 bytes, para un total de 17 bytes. El cliente lo envía después de procesar y dibujar el CHUNK. Si llega un duplicado, el cliente evita contabilizarlo otra vez y puede repetir el ACK si ya había dibujado ese chunk.

CHUNK_REF

CHUNK_REF contiene generationId (8), índice (4), columna y fila (4 cada una), posición en el canvas X/Y (4 cada una) y ancho/alto (4 cada uno). No transporta bytes de imagen: el navegador recupera el bitmap de su propia caché y lo dibuja en la posición indicada.

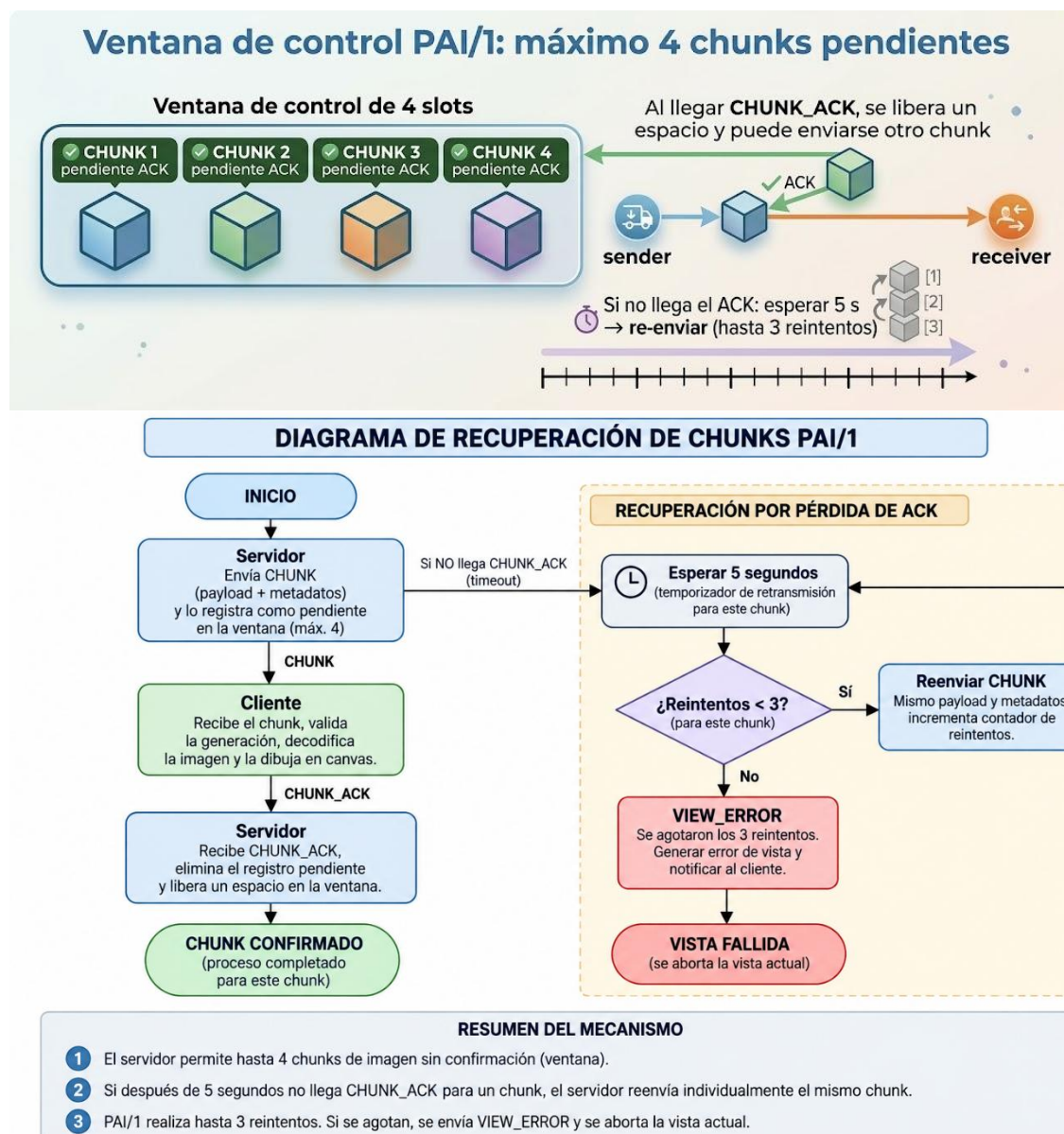
VIEW_END

VIEW_END contiene generationId (8 bytes), cantidad de piezas (4) y suma de payloads de imagen únicos de la vista (8). El servidor espera a que no queden CHUNK sin ACK; las retransmisiones no se suman a imageBytes.

VIEW_ERROR

VIEW_ERROR contiene generationId (8), longitud del texto (2) y una descripción UTF-8 de hasta 240 bytes. Se utiliza cuando una vista falla; el código intenta notificar el error sin cerrar la conexión WebSocket si todavía puede escribir en ella.

Control de flujo y recuperación



Ventana deslizante

ViewFlowControl limita los chunks de imagen enviados que todavía no han recibido CHUNK_ACK. Si hay cuatro pendientes, el servidor espera a que llegue una confirmación o a que venza el temporizador de recuperación. Esto evita que el servidor siga acumulando indefinidamente chunks no confirmados para un mismo cliente.

Parámetro	Valor en el código	Efecto
Ventana	4 chunks	Máximo de chunks pendientes de ACK por sesión y generación.
Temporizador de reenvío	5 segundos	Si el chunk sigue pendiente tras este intervalo, se vuelve a enviar.
Reintentos máximos	3	Tras alcanzar el límite sin ACK, la vista falla.
ACK tardío	Se tolera si el índice ya fue confirmado	Evita tratar como error un ACK repetido de un reenvío.
Fin de vista	Espera a ventana vacía	VIEW_END no se envía mientras queden chunks de imagen pendientes.

La ventana de 4 y el timeout de 5 s son límites fijos de la implementación para acotar trabajo pendiente y recuperar un ACK no recibido; no son valores óptimos demostrados por una comparación de rendimiento. Cada CHUNK pendiente se reenvía individualmente, hasta 3 veces. TCP mantiene su propio control de congestión.

Qué pasa si se pierde una confirmación

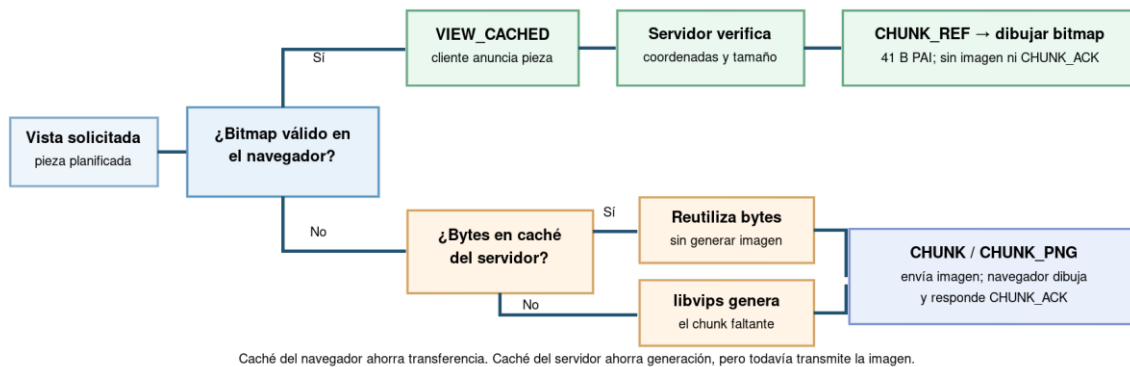
- El servidor envía un CHUNK y lo registra como pendiente.
- El navegador recibe el mensaje, valida los datos, decodifica la imagen y la dibuja.
- El navegador envía CHUNK_ACK con la misma generación y el mismo índice.
- Si el servidor no recibe el ACK dentro de cinco segundos, reenvía el mismo payload.
- Si llega un duplicado al navegador, este no vuelve a contarlos como una pieza nueva; si ya lo dibujó, puede volver a enviar el ACK.
- Si el límite de reintentos se agota, el servidor marca la vista como fallida y trata de notificar VIEW_ERROR. El ACK de PAI confirma que el cliente procesó el chunk de imagen; no reemplaza las confirmaciones internas de TCP. La implementación actual aplica el flujo de ACK a CHUNK y CHUNK_PNG. Las referencias CHUNK_REF no contienen bytes de imagen y se envían por una ruta distinta, sin pasar por la ventana de CHUNK_ACK.

Generaciones y cancelación

generationId distingue una vista de otra. Cuando el usuario hace zoom o mueve la imagen, el navegador genera una nueva solicitud con un identificador mayor. ViewCoordinator cancela la tarea anterior si todavía está activa. Tanto el servidor como el navegador comprueban la generación antes de continuar enviando, procesando o dibujando.

Esto reduce trabajo desperdiciado y evita que fragmentos atrasados de una vista vieja se mezclen visualmente con la vista nueva. La cancelación no es un mensaje PAI independiente; es una política interna coordinada por el identificador de generación y la tarea activa.

Dos cachés, dos ahorros distintos (por chunk)



Caché del servidor: GreedyDual-Size

ChunkCache guarda bytes codificados de PNG/JPEG, con un presupuesto global de 32 MiB. En un acierto refresca la prioridad $H = \text{edad} + \text{costo_ms}/(\text{tamaño_bytes}/\text{MiB})$; al faltar espacio expulsa la menor H y actualiza edad con esa prioridad. El costo de un lote generado se estima como tiempo de preparación más codificación dividido entre sus chunks. Solo publica el lote después de generarlo completo.

La clave del servidor incluye ID, tamaño y fecha del original, zoom, dimensiones del nivel, tamaño de chunk, columna y fila. Así no reutiliza bytes de otra versión o geometría. $\text{cache.maxMiB}=32$ limita esta caché, no toda la memoria del proceso.

Caché del navegador

El navegador conserva bitmaps decodificados con presupuesto estimado de 12 MiB ($\text{ancho} \times \text{alto} \times 4$). La prioridad usa $\text{edad} + \text{tiempo_de_decodificación}/(\text{bytes}/\text{MiB})$. Anuncia como máximo 128 piezas compatibles en VIEW_CACHED y las fija hasta terminar la vista para que no sean expulsadas antes de un CHUNK_REF.

La clave del navegador incluye imagen, tamaño del original, zoom, viewport, columna, fila y dimensiones de pieza. Al cambiar de imagen se liberan las piezas fijadas; las antiguas pueden permanecer hasta expulsión por presupuesto. Al renovar el catálogo o desconectar se vacía la caché. CHUNK_REF ahorra transferencia de imagen; un acierto del servidor evita generación pero todavía envía CHUNK y requiere ACK.

Procesamiento de imágenes y chunks

Cómo se calcula y entrega una vista

Escala: $s_0 = \min(1, Vw/W, Vh/H)$. Cada nivel duplica s hasta 1:1; si algún lado del original mide al menos 50 000 px, se agregan $2\times$ y $4\times$. Esos dos últimos niveles amplían píxeles, no crean detalle.

Región: se redondea $W\times s$ y $H\times s$, se proyecta el centro solicitado y se ajusta el rectángulo visible para no salir de la imagen. Con chunks de lado C , las columnas son $\lfloor x/C \rfloor$ a $\lfloor (x+w-1)/C \rfloor$; las filas se calculan igual. La cuadrícula global mantiene estable la identidad de cada pieza al mover la vista.

De una VIEW a seis chunks

1. Escala

$s = \min(1, Vw/W, Vh/H)$; luego $\times 2$

2. Región visible

Centro proyectado y ajustado a los bordes

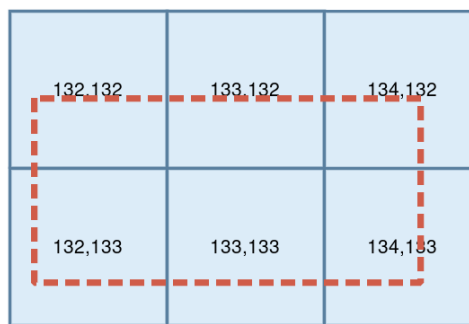
3. Cuadrícula global de 512 px

columnas = $x/512 \dots (x+w-1)/512$

Ejemplo medido: imagen 055

1:1; viewport 1000 $\times$ 600; centro 68 162
columnas 132–134, filas 132–133 = 6 chunks

Piezas seleccionadas



Rojo: viewport; azul: tiles completos reutilizables

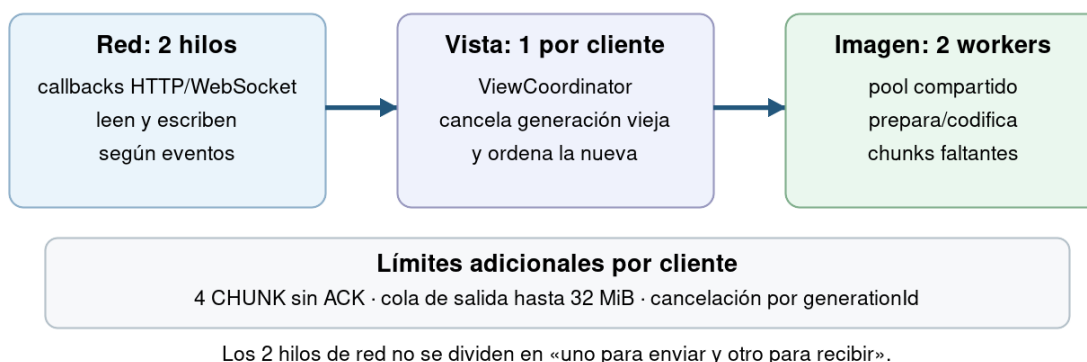
Ejemplo real: para la imagen 055, con viewport 1000 $\times$ 600, centro (68 162, 68 162) y escala 1:1, la región es (67 662, 67 862) de 1000 $\times$ 600; intersecta columnas 132–134 y filas 132–133: seis piezas. El último tile de un borde se recorta al tamaño de la imagen.

Después de planificar: el servidor separa referencias anunciadas por el cliente, aciertos de bytes en su caché y faltantes. VIEW_START sale antes de preparar píxeles. Solo para los faltantes calcula el rectángulo mínimo que los contiene, resuelve la fuente (vista general, BigTIFF o original) y usa libvips. Los workers entregan piezas al completarse; a escala inferior a 1:1 codifica JPEG y desde 1:1, PNG.

- Para originales gigantes que no admiten lectura regional eficiente, make prepare-image crea una vista general JPEG y un BigTIFF mosaico de resolución completa, sin pirámide. make run no prepara automáticamente. La vista general se usa solo si cubre la escala solicitada; la ruta de detalle usa el mosaico. La conversión inicial puede requerir varios GiB temporales.

Hilos, límites y decisiones

Concurrencia: tres responsabilidades distintas



server.threads=2 configura el grupo de callbacks de red; ninguno queda reservado exclusivamente para enviar o recibir. Cada conexión tiene un ViewCoordinator de un hilo que cancela su VIEW anterior. view.workers=2 configura un pool compartido que prepara y codifica chunks faltantes. Son tres responsabilidades, no dos workers con dos hilos cada uno.

Dos hilos de red, dos workers, una ventana de cuatro CHUNK y una cola de salida de 32 MiB por cliente son límites iniciales para contener concurrencia y memoria; este documento no afirma que sean el óptimo de rendimiento. El presupuesto de cola es por cliente, mientras que los 32 MiB de ChunkCache son globales.

Configuración

Clave	Valor en config/server.properties	Descripción
port	8080	Puerto del servidor.
chunk.size	512	Lado del chunk en píxeles; rango admitido 128–1024.
server.threads	2	Hilos de servidor; rango admitido 1–32.
view.workers	2	Workers para vistas/imágenes; rango admitido 1–32.
cache.maxMiB	32	Presupuesto de la caché de chunks; rango admitido 1–4096 MiB.
writeQueue.maxMiB	32	Presupuesto de cola de escritura por cliente; rango admitido 16–4096 MiB.
jpeg.normalQuality	85	Calidad JPEG para zooms inferiores a 1:1; rango admitido 1–100.

Opciones del navegador (web/config.js)

Opción	Valor	Efecto
enableExtraGiantZoom	true	Muestra 2× y 4×; el servidor sigue aceptándolos.
bitmapCacheMiB / decodeLanes	12 / 2	Límite estimado de bitmaps / decodificaciones paralelas.
maxReconnectAttempts	5	Máximo de intentos de reconexión WebSocket.

reconnectBaseDelayMs / reconnectMaxDelayMs	500 / 8000	Espera exponencial mínima y máxima.
wheelDebounceMs	150	Espera tras la rueda antes de solicitar otra VIEW.

Los valores de ambas tablas son los predeterminados incluidos, no límites universales de PAI/1. Las medidas del formato y la ventana de ACK están fijadas en código. Cambiar web/config.js requiere recargar la página; cambiar server.properties requiere reiniciar Java.

Validación y robustez

- Comprueba la firma PAI, la versión y que el opcode sea conocido y esperado.
- Valida que las longitudes declaradas correspondan al tamaño real del mensaje.
- Rechaza VIEW incompletos, IDs inválidos, índices de chunk fuera de rango y metadatos de imagen inconsistentes.
- El servidor requiere frames WebSocket binarios, enmascarados por el cliente y no fragmentados en esta implementación.
- Una operación PAI desconocida o una cabecera inválida genera un error de protocolo; no debe interpretarse como una solicitud válida.
- La cola de salida por cliente está limitada por bytes para evitar que un cliente lento provoque crecimiento ilimitado de mensajes pendientes.

Referencias técnicas

WebSocket

RFC 6455, *The WebSocket Protocol*. Define el protocolo utilizado para establecer una comunicación bidireccional entre navegador y servidor. En este proyecto, WebSocket proporciona el canal de transporte sobre el cual se intercambian los mensajes binarios PAI/1.

TCP

RFC 9293, Transmission Control Protocol (TCP). Define el transporte confiable utilizado debajo de WebSocket. Los mecanismos de confiabilidad propios de TCP son independientes del mecanismo de ACK implementado por PAI/1.

Control de flujo y recuperación

Los conceptos de ventana, confirmación, retransmisión y recuperación se basan en mecanismos conocidos de protocolos de transferencia confiable. PAI/1 implementa una política propia de ventana de cuatro chunks, ACK y retransmisión por timeout; no pretende ser una implementación completa de Go-Back-N ni Selective Repeat.

Relación con PAI/1

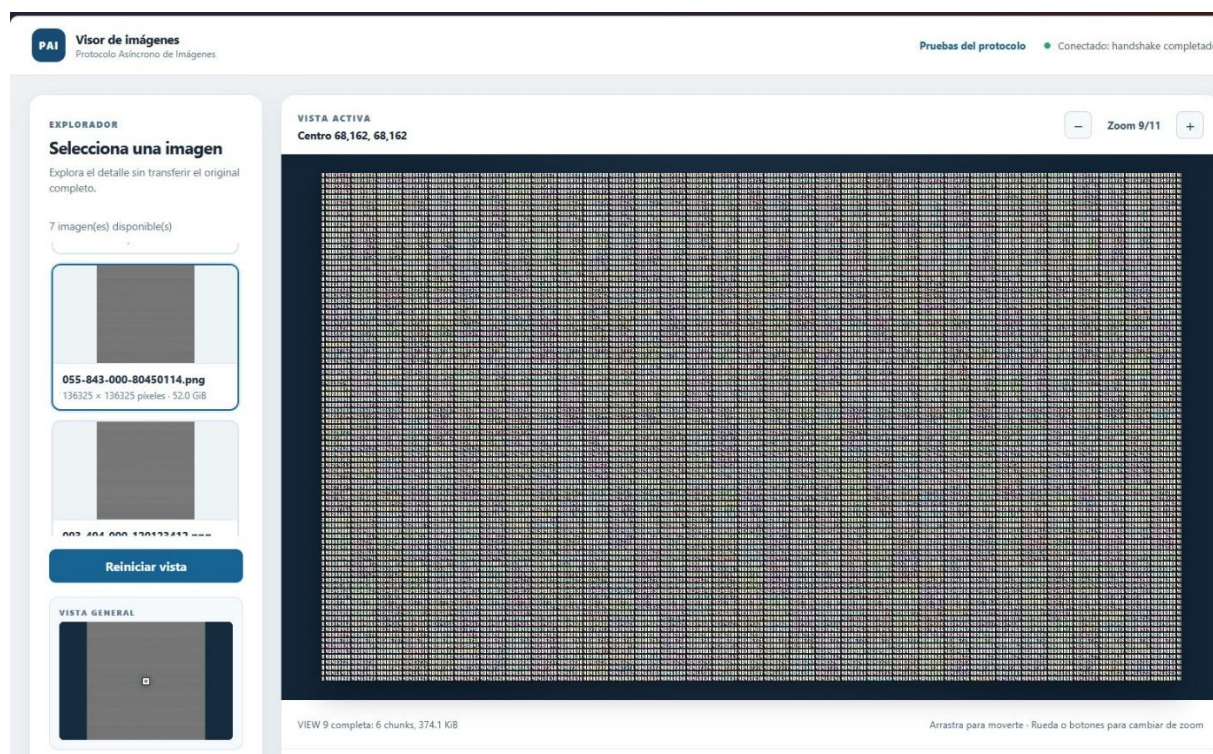
WebSocket y TCP proporcionan mecanismos de comunicación y transporte; PAI/1 define los mensajes específicos para solicitar vistas, transferir chunks, confirmar su procesamiento y reutilizar fragmentos almacenados en caché.

Evidencia de evaluación

Se realizaron pruebas utilizando imágenes de gran tamaño para comprobar que PAI/1 permite visualizar regiones de imágenes de decenas de gigabytes sin transferir el archivo original completo al navegador.

Prueba con imagen de 52.0 GiB

La primera prueba se realizó con la imagen:



La captura incluida muestra zoom 9/11 y una vista completada con 6 chunks y 374.1 KiB de payloads de imagen. Esta captura no demuestra por sí sola el zoom 11/11.

Esto evidencia que el cliente recibe únicamente los fragmentos necesarios para la región visualizada, en lugar de recibir los 52.0 GiB de la imagen original.

Prueba con imagen de 87.1 GiB

También se realizó una prueba con la imagen:



La captura incluida muestra zoom 10/12 y una vista completada con 6 chunks y 345.0 KiB de payloads de imagen.

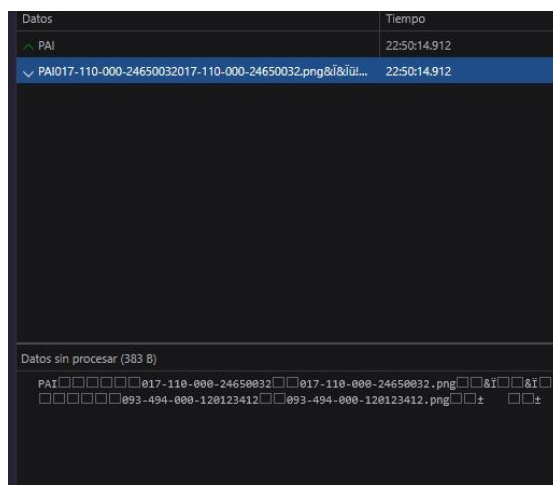
Al igual que en la prueba anterior, la cantidad de datos transferidos para cada vista es muy inferior al tamaño del archivo original, ya que PAI/1 selecciona y transmite únicamente los fragmentos requeridos por el viewport.

Evidencia del protocolo mediante WebSocket

La conexión se realiza mediante el endpoint:

ws://localhost:8080/pai

Se muestra la conexión WebSocket /pai y un payload binario. Cada payload PAI comienza con Magic, versión y opcode.



Medición controlada de primera vista y repetición

Se realizó una prueba sobre `ws://127.0.0.1:18080/pai`, con viewport 1000×600, los centros indicados abajo y una misma conexión. Envío VIEW, respondió CHUNK_ACK a cada CHUNK_PNG y repitió la vista con VIEW_CACHED anunciando las seis geometrías recibidas. Se midió desde el envío de VIEW hasta VIEW_END; el cliente no decodifica ni dibuja bitmaps.

Imagen / centro / zoomIndex	Primera VIEW	Repetición VIEW_CACHED	Payload de imagen
055 / (68 162, 68 162) / 8	6390 ms · 6 CHUNK_PNG	47 ms · 6 CHUNK_REF	383 090 B → 0 B
093 / (88 196, 88 196) / 9	484 ms · 6 CHUNK_PNG	44 ms · 6 CHUNK_REF	353 266 B → 0 B

Trama observada en ambos casos: primera vista = VIEW_START (45 B) → 6 CHUNK_PNG → VIEW_END (25 B), con 6 CHUNK_ACK del cliente; repetición = VIEW_START (45 B) → 6 CHUNK_REF (41 B cada uno) → VIEW_END (25 B), sin CHUNK_ACK. Los tamaños son de payload PAI, no incluyen encabezados WebSocket/TCP.

Peor tiempo de estas cuatro vistas: 6390 ms.

Compilar, ejecutar y preparar imágenes

Desde la carpeta raíz del proyecto en Linux o WSL:

```
make check-tools
make compile
make run
```

El servidor anuncia la página local en el puerto configurado, por defecto `http://localhost:8080/`, y el endpoint WebSocket es `ws://localhost:8080/pai`.

Para preparar una imagen gigante cuyo formato lo requiera:

```
make prepare-image IMAGE=ID_DE_LA_IMAGEN
```

Para generar únicamente la vista general:

```
make prepare-overview IMAGE=ID_DE_LA_IMAGEN
```

El ID debe coincidir con el identificador que el catálogo asigna a la imagen. La preparación detallada puede necesitar varios GiB de espacio temporal y no se ejecuta al iniciar el servidor.